

DD2459 Software Reliability 2023
Take-home Exam
Tuesday 7 March, 15.00 – Thursday 9 March, 12.00 midday
(Version 1.0)

Instructions.

Please read the following instructions carefully.

- (1) Only hard-copy exam submissions are accepted. Clearly mark at the top of each sheet of paper you use: (a) your name, (b) the page number.
- (2) On your front page indicate: (a) how many pages are contained in your work in total, (b) your name (c) your personal or KTH number, (d) your e-mail address (in case I need to contact you)
- (3) There are two ways to **submit** your work. (a) Your work may be **handed in personally by you only** to EECS studentexpedition, E building, level 4, Lindstedsvägen 3, no later than Thursday 9 March 2023 at 12.00 midday. After this time, it will be marked as late, and marks will be subtracted. **Please take identification with you if you submit this way, e.g. a legitimization or passport.** (b) If you are unable to reach the studentexpedition yourself (for example if you are at work) you may **post** your manuscript to: Karl Meinke, EECS School, KTH Kungliga Tekniska Högskolan, 100 44 Stockholm, Sweden. The date on the postmark will be taken as the date of your submission. The deadline of March 9, 2023 applies to all manuscripts submitted by post.
- (4) If manuscripts are submitted in any other place or by any other means than those described in Part (3) then the examiner and EECS School cannot be held responsible in the case that manuscripts are lost. In the case of postal submission, it is strongly recommended that you keep a digital copy or photocopy in case of postal loss. KTH cannot be held responsible for loss in any national postal system.
- (5) If you have any questions about the exam (for example, if you do not understand a question) you may call the examiner on 08 790 6337. Please do not call before 10.00 am or after 5.00pm, and be aware that I may be in a meeting! You can also e-mail the examiner at karlm@kth.se.
- (6) I will publish any typographical errors and corrections that become known during the exam on the course exam web page and notify you with Canvas. Therefore, you should check back regularly on the course exam page for updates that will have new version numbers 1.x.
- (7) You may use your course notes, books and the internet. However, all material you submit must be your own. (a) You are not allowed to discuss your answers with anyone else until the exam is finished. (b) You are not allowed to copy anyone else's work. (c) By handing in your manuscript you are declaring that you have read and abide by all the rules on this cover page. (d) In the case that cheating is suspected, disciplinary action will be taken.
- (8) Hand-written answers must be written clearly. No marks will be awarded for work that I cannot read. You can write your answers in Swedish or in English. You can submit a hand-written manuscript or a typed manuscript or any combination of both.

NOTE: For a full score of 80 points you should answer all 4 questions below.

Question 1. (Total 30 points)

Write appropriate pre- and post-conditions for the following mathematical operations using the JML specification language (i.e. write *requires-ensures* conditions).

You should try to avoid under-specification, i.e. write all the constraints necessary for each condition and do not omit any relevant constraints.

If you are unsure how any mathematical operation/method is defined you may look it up online or in a book.

You do not have to provide Java code for any of the operations, only JML.

- (i) **(5 points)** An *identity matrix generator* for 2-dimensional floating-point valued matrices:

```
float[][] identity(int n)
```

Input: A non-zero integer size parameter n .

Output: The identity matrix of size n (commonly termed I_n in maths books).

Model Answer:

Following the standard mathematical definition:

```
requires n > 0;
```

```
ensures
\result != null &
\result.length == n &
(\forallall int i; 0 <= i < n;
    \result[i] != null & \result[i].length == n) &
(\forallall int i; 0 <= i < n;
    (\forallall int j; 0 <= j < n;
        i == j => \result[i][j] == 1.0 &
        i <> j => \result[i][j] == 0.0 &
    )
)
```

- (ii) **(10 points)** A *matrix inversion function* for floating point valued matrices:

```
float[][] invert(float[][] A)
```

Input: A matrix A of floating-point values.

Output: The matrix inverse of A .

Hint: You may use the method

float[][] identity(int n)

of part (i) in your answer. You should avoid any use of determinants which are very complicated to define for $n > 3$.

Model Answer:

```
requires
// A is a well-defined square matrix
A != null &
A.length > 1 &
(\forall int i; 0 <= i < A.length;
    A[i] != null & A[i].length == A.length) &
// A has an inverse, i.e. A is non-singular
(\exists float[][] B;
// B is a well-defined square matrix of the same size as A
B != null &
B.length == A.length &
(\forall int i; 0 <= i < A.length;
    B[i] != null & B[i].length == A.length))
&
// A.B == I_n where n = A.length
(\forall int i; 0 <= i < A.length;
    (\forall int j; 0 <= j < A.length;
        ((\sum int k; 0 <= k < A.length;
            A[i][k]*B[k][j] = identity(A.length)[i][j]
        )
    )

ensures
// \result is the inverse of A

\result != null &
\result.length == A.length &
(\forall int i; 0 <= i < A.length;
    \result[i] != null &
    \result[i].length == A.length))
&
// A. \result == I_n where n = A.length
(\forall int i; 0 <= i < A.length;
    (\forall int j; 0 <= j < A.length;
        ((\sum int k; 0 <= k < A.length;
            A[i][k]* \result[k][j] = identity(A.length)[i][j]
        )
    )
```

- (iii) **(15 points)** Comment on the practical usefulness of your JML precondition in part (ii) to generate test cases for a Java implementation of matrix inversion. Can you see any possibility to improve your JML answer to make a more efficient solution for test case generation? Write out a test script based on your more efficient proposal that combines random testing with your JML requirement model to achieve both:

(a) automated valid test case generation and (b) automated verdict construction (the oracle step).

Model Answer: The solution above avoids explicit use of the determinant $\det(A)$ of A , which would have to be defined in JML and has a complex formula for $n > 3$. We recall that A is invertible if, and only if its $\det(A)$ is non-zero. Avoiding all mention of $\det(A)$ makes it hard to check whether there really exists a B that is inverse to test case A . In this respect the precondition is not helpful or efficient to find test cases which are legitimate test cases.

A more complicated improvement would be to encode determinants in JML such as the Leibniz formula or Laplace expansion for $\det(A)$. However, evaluating the determinant would then filter out any non-invertible test cases.

Furthermore, randomly chosen singular matrices are fairly rare on average, so the suggested improvement would be efficient when applied to randomly generated test matrices, and the overhead of filtering out bad test data would be very small.

An efficient JML-based test script that uses an implementation of $\det(A)$ would be

```
tested = false;
repeat until (tested) {
    // choose a matrix size n randomly
    n = random(int);

    // build random matrix of size n
    for i = 1 to n do
        for j = 1 to n do
            A[i][j] = random(float);

    // Check if A is invertible, and if so
    if (det(A) <> 0.0) {

        // call SUT
        B = invert(A);

        // evaluate the postcondition i.e. test oracle

        for i = 1 to n do
            for j = 1 to n do {

                // calculate inner product
                innerproduct = 0;
                for k = 1 to n do
                    innerproduct =
                        innerproduct + A[i][k]*B[k][j];

                // Ignoring an epsilon error term below
                if (innerproduct <> identity(n)[i][j])
                    then
```

```

                                return fail;
                            } // for j = ...

                            // If we get this far we have generated
                            // and executed one valid test case

                            tested = true;

                        } // if

                    return pass;

                } // repeat until

```

Notice this script is not guaranteed to terminate, but does so with high probability! Note the innerproduct value could be extremely small but non-zero, so the oracle step should probably have an epsilon error term, but this is omitted for clarity!

Question 2. (Total 10 points)

- (i) (6 points) Write down two different metamorphic equations that should be satisfied by any function:

```
boolean contains(int[] myIntList, int key)
```

with

Input: A one-dimensional array `myIntList` of `int` and an integer `key` .

Output: `true` if `myIntList` contains `key` otherwise `false`.

Clearly define the data transformation T used on the input for each of your two equations.

Model Answer: The `contains` function satisfies any permutative law (similar to the inclusive law mentioned in Lecture 6) such as

$$\text{contains}([x_1, \dots, x_n], y) = \text{contains}(\text{perm}([x_1, \dots, x_n], y))$$

where perm is any permutation, for example:

$\text{perm}([x_1, \dots, x_n], y) = ([x_n, \dots, x_1], y)$ is the reversal permutation

$\text{perm}([x_1, \dots, x_n], y) = ([x_n, x_1, \dots, x_{n-1}], y)$ is the shift-1 right permutation

The `contains` function also satisfies any extension function

$$\text{contains}([x_1, \dots, x_n], y) = \text{contains}(\text{extend}([x_1, \dots, x_n], y))$$

where extend is any extension, for example:

$\text{extend}([x_1, \dots, x_n], y) = ([x_1, \dots, x_n, x_n], y)$ is a right extension

$\text{extend}([x_1, \dots, x_n], y) = ([x_n, x_1, \dots, x_n], y)$ is a left extension

There are many variations on extension such as list doubling

$$\text{double}([x_1, \dots, x_n], y) = ([x_1, \dots, x_n, x_1, \dots, x_n], y)$$

You may be able to find other laws that hold?

- (ii) **(4 points)** Starting from one concrete seed test case (i.e. input values), write down as many different test cases as you can generate using the two data transformations T, T' that you defined in your answer to part (i). Is there any finite limit on the number of different test cases you can generate in either case? If so, explain what is the limiting factor.

Model Answer: Given a seed test case such as $([1, \dots, n], y)$ a new test case with reversal would be

$$T([1, \dots, n], y) = ([n, \dots, 1], y).$$

However, if we try to apply the reversal permutation T twice then we get

$$T^2([1, \dots, n], y) = ([1, \dots, n], y).$$

So higher powers of reversal $T^n([1, \dots, n], y)$ are just one of these two vectors and we only get two distinct test cases.

Notice that shift-1 right gives n new test cases and in general other permutations give different numbers of test cases, but no fixed permutation gives more than n . Note also that some permutations can give no new tests, depending also on the seed test, e.g. reversal of a symmetric vector!

If the extension transformation is used then there is no limit to the number of test cases that can be generated. Given a seed test case such as $[1, \dots, n]$ a new test case with extension would be e.g.

$$T^n([1, \dots, n], y) = ([1, \dots, n, a, \dots, a], y)$$

where a is repeated n times.

Question 3. (Total 15 points)

Figure 1 shows a 4-way road junction where a side road (heading north and south) cuts across a main road (heading east and west). The main road usually carries more traffic than the side road. Therefore, a traffic light is installed at the junction to allow all traffic to access the junction in a controlled manner. The requirements on the traffic light (from the perspective of the main road drivers) are defined by:

- (1) *The main road lights (facing east and west) stay green unless a car is sensed on the side road, or a pedestrian tries to cross the main road by pushing a request button.*
- (2) *From green, the main road lights turn amber for two time-units*
- (3) *From amber the main road lights turn red.*
- (4) *The main road lights are red for at least two time-units, and remain red so long as a car is sensed on the side road, or a pedestrian has requested to cross the main road.*
- (5) *From red, the main road lights turn red&amber (both colours) for one time-unit*
- (6) *From red&amber the main road lights turn green.*

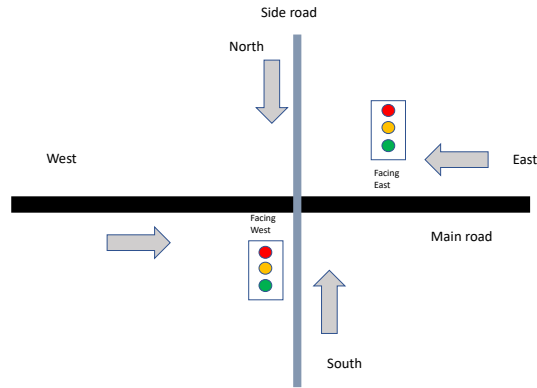
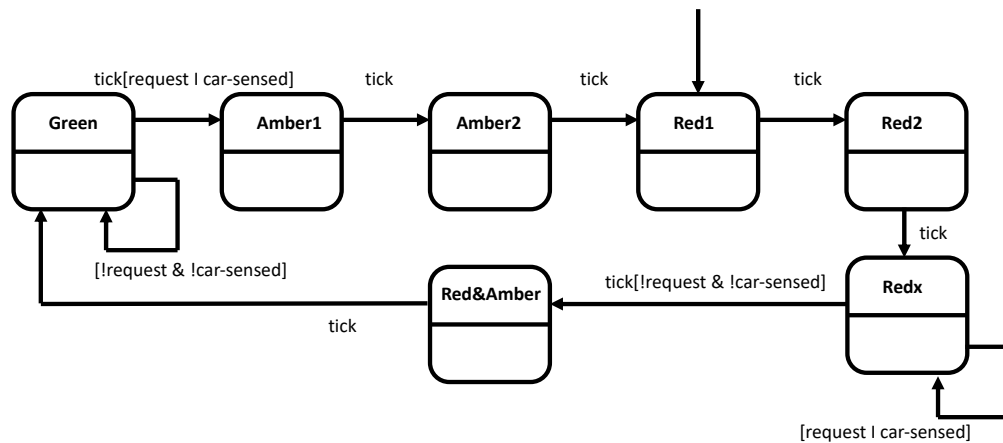


Figure 1 Traffic Light at Crossroads

- (i) **(4 points)** Draw a UML state machine (also known as a statechart) which models the main road traffic light facing west (see Figure 1). Make sure to use UML events, conditions and actions correctly in your model. Your model should use a clock that generates a regular *tick* event after each unit time interval, and Boolean variables for the request button and car-sensor.

Model Answer:



- (ii) **(2 points)** Write a set of LTL trap properties which give 100% edge coverage for your model in answer to part (i).

Model Answer: There are 9 edges and hence 9 trap properties:

```

G( state = Green & (request | car_sensed) => X(!state = Amber1) )
G( state = Green & (!request & !car_sensed) => X(!state = Green) )
G( state = Amber1 & tick => X(!state = Amber2) )
G( state = Amber2 & tick => X(!state = Red1) )
G( state = Red1 & tick => X(!state = Red2) )
G( state = Red2 & tick => X(!state = Redx) )
G( state = Redx & (request | car_sensed) => X(!state = Redx) )
G( state = Redx & (!request & !car_sensed) => X(!state = Red&Amber) )
G( state = Red&Amber & tick => X(!state = Green) )
  
```

- (iii) (3 points) After just a few weeks, some drivers began to complain to the city planning authority that this traffic light system was unfair. Which drivers complained and what did they say? Use your answer to part (i) in your explanation.

Model Answer: From the statechart above we can see that once the side road gets a green light (i.e. main road light state is Redx) then the presence of traffic on the side road can shut out the main road indefinitely. This would cause the main road drivers to complain at times of high side road traffic. This is reflected by the fact that the statechart is not symmetric in its design between states Green and Redx (i.e. between NS and EW) which it probably should be.

- (iv) (2 points) Using your answer to part (i), write down an LTL formula that models the unfairness situation which the drivers in part (iii) complain about.

Model Answer:

The *fairness* of the traffic light from the main road drivers' perspective is that they can always eventually leave state Redx by getting a green light. In LTL this is:

$$G(\text{state} = \text{Redx} \Rightarrow F(\text{state} = \text{Green}))$$

More generally, we might say that they are never stuck indefinitely in any specific state, e.g.

$$G(\text{state} = "x" \Rightarrow F(\text{state} \neq "y"))$$

(for each state "x" where "y" is the follow on state to "x").

The unfairness condition is therefore the negation of this i.e.

$$\neg G(\text{state} = \text{Redx} \Rightarrow F(\text{state} = \text{Green})).$$

Using De Morgans laws this is logically equivalent to

$$F(\text{state} = \text{Redx} \ \& \ G(\neg \text{state} = \text{Green})).$$

Notice that a statement such as:

$$G(\text{state} = \text{Redx} \ \& \ (\text{request} \mid \text{car_sensed}) \Rightarrow X(\text{state} = \text{Redx})) \ (*)$$

simply describes the model. If we negate this formula to:

$$\neg G(\text{state} = \text{Redx} \ \& \ (\text{request} \mid \text{car_sensed}) \Rightarrow X(\text{state} = \text{Redx})) \)$$

Then using De Morgan's laws, this is logically equivalent to:

$$F(\text{state} = \text{Redx} \ \& \ (\text{request} \mid \text{car_sensed}) \ \& \ X(\text{state} \neq \text{Redx})) \ (**)$$

However, (**) does not mean that the model is always fair, only that it is sometimes fair. For example, maybe we only eventually get out of Redx the first time, but never again, which would still be fundamentally unfair.

Also, it's not clear that Red&Amber (or any other later state such as Green) necessarily follows after Redx.

Finally (**) says that sometimes request buttons or cars on the side road will not be granted access, so an implementation could never give access and yet satisfy this formula!

Lastly (**) does not forbid that we could get stuck in Redx some other way.

So, (*) and equivalently (**) are not unfairness as the negation of fairness. Two lessons here: (1) negating over infinite time is tricky! (2) don't try to reverse engineer requirements from implementations (or models!).

- (v) **(4 points)** Is it possible to model-based test an implementation of these traffic light requirements for fairness? Explain your reasoning in detail, using your answer to part (iv).

Model Answer: From the formula

$$F(\text{state} = \text{Redx} \ \& \ G(\neg \text{state} = \text{Green}))$$

we can see that the unfairness situation could arise at any time in the future. Furthermore, the traffic light can stay Redx for an arbitrary amount of time without being unfair. These two aspects each individually mean that there is no way to stop a test case for fairness after some finite time and conclude that the traffic light is actually fair, i.e. issue a test verdict *pass*.

(Please turn over)

Question 4. (Total 25 points)

Harry Hopeful wanted to unit test the following code

```
myPoly(int x, y) {
    int result, w, i, j;

    result = 0;
    w = 0;
    i = 1;
```

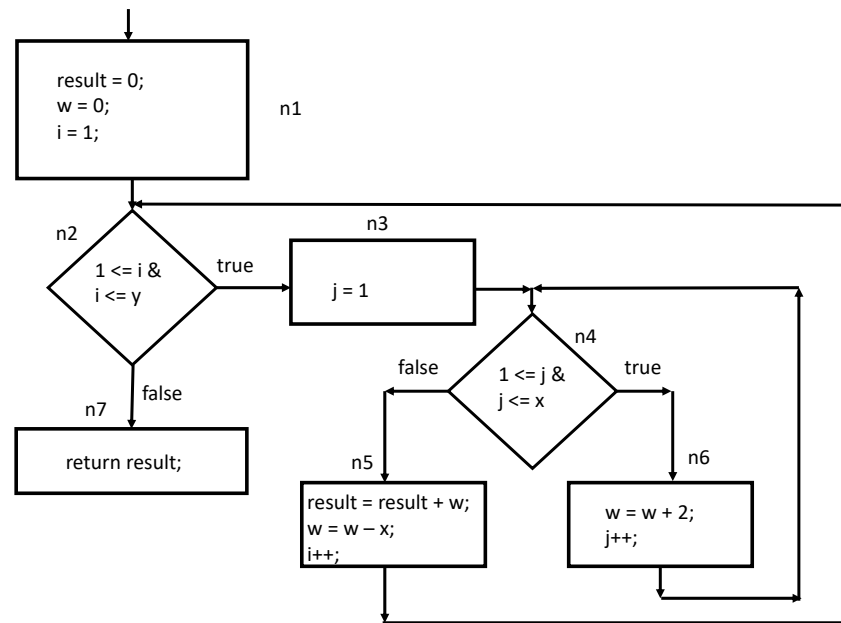
```

while (1 <= i <= y) {
    j = 1;
    while (1 <= j <= x) { w = w + 2; j++;}
    result = result + w;
    w = w - x;
    i++;
}
return result;
}

```

(i) (2 points) Draw a condensation graph for Harry's code.

Model Answer:



(ii) (5 points) Harry was particularly interested in testing the loops in this code, and was able to recall the graph theoretic concepts of a simple path and a prime path. Write out the set of all prime paths for the condensation graph that is your answer to part (i) above.

Model Answer: There are exactly 13 prime paths for the condensation graph above

P1 = n1, n2, n7
 P2 = n1, n2, n3, n4, n5
 P3 = n1, n2, n3, n4, n6
 P4 = n2, n3, n4, n5, n2
 P5 = n3, n4, n5, n2, n3
 P6 = n3, n4, n5, n2, n7
 P7 = n4, n6, n4
 P8 = n4, n5, n2, n3, n4
 P9 = n5, n2, n3, n4, n5

P10 = n5, n2, n3, n4, n6
 P11 = n6, n4, n6
 P12 = n6, n4, n5, n2, n3
 P13 = n6, n4, n5, n2, n7

- (iii) (4 points) For a given condensation graph G , we say that a glassbox control flow test requirement (i.e. a path in G)

$$TR_x = p = v_1, \dots, v_k$$

implies another control flow test requirement (i.e. a path in G)

$$TR_y = q = v'_1, \dots, v'_j,$$

if, and only if, every test case TC for G that satisfies TR_x also satisfies TR_y .

Looking over the prime paths of your answer to part (ii) as test requirements, write down all the pairs of prime paths (p, q) such that p implies q .

Model Answer: Due to the structure of the condensation graph, there are 3 main clusters of pairs

- (1) (2,4), (6,4), (8,4), (9,4)
- (2) (8,5)
- (3) (3,7), (10,7), (11,7)

Note that some more implications exist based on a deep semantic analysis of the code, but I did not assume this for a basic answer and bonus points were awarded for anyone who found them.

- (iv) (4 points) Using your answers to parts (ii) and (iii), write out a minimal set of control flow test requirements to achieve 100% prime path coverage (PPC). Explain your answer.

Model Answer: From our answer to (iii), we can see that the prime paths P4, P5 and P7 are redundant as test requirements, as they are implied or automatically covered by at least one other prime path in the set. So, a minimal set of test requirements for 100% PPC would be:

PPC TR1 = P1
 PPC TR2 = P2
 PPC TR3 = P3
 PPC TR4 = P6
 PPC TR5 = P8
 PPC TR6 = P9
 PPC TR7 = P10
 PPC TR8 = P11

- (v) (2 points) Harry was told that the functional requirements for his code were the following JML pre and postconditions:

requires $x \geq 0 \ \& \ y \geq 0$

ensures $\text{result} == 2 * \text{old}(x) * \text{old}(y)$

Using his test suite, Harry found that the code contained an error on just one single line of code. **(a)** Write down the line of code containing the error, and explain why this line is incorrect with respect to the JML requirement above? **(b)** How could you correct the program, so that it satisfies the JML requirement, by changing this single line only?

Model Answer: The error is the assignment statement

`w = w - x;`

which fails to re-initialise `w` to 0 for further transitions of the innermost loop. A suitable correction based on this line alone is

`w = 0;`

or alternatively

`w = w - 2*x;`

- (vi) (8 points)** To his surprise, Harry realized that only one prime path test requirement was guaranteed to reveal the single line coding error of part (v). All his other prime path test requirements were either irrelevant or could miss the error due to an unlucky choice of his satisfying test case. (Harry did not think himself a lucky tester!)

Which single prime path test requirement tr is guaranteed to find the error for any test case tc that is chosen to satisfy tr ? Carefully explain your answer with reasoning about the code structure. What can you conclude about the ability of 100% PPC coverage to find errors in loops compared with other loop testing strategies that you know of?

Model Answer: The prime path test requirement

PPC TR7 = P10 = `n5, n2, n3, n4, n6`

is guaranteed to find the error of part(v), as any test case that satisfies TR7 must execute the innermost code loop twice in succession, thus revealing the incorrect re-initialisation of `w` in between.

This is because any test case

`tc = (x = a, y = b)`

satisfying P10 must cause execution of node `n6` at the end of P10. From the code, this implies that immediately previously at `n4` `x >= 1`. However variable `x` is never modified by any program instruction, so `b >= 1` and `x` is always `>= 1`.

Just before entering node `n5` of P10, execution of `tc` must have passed through node `n4`, because there is no other way to reach `n5`. When `n4` is passed through at this time it must be because the innermost loop has also just previously been taken (as `x >= 1` is always true). Now `w = 2x` after executing the innermost loop on this previous occasion. Then after node `n5` `w = 2x - x` and since `x >= 1` then `w != 0`. Thus the assignment `w = 2x - x` does not re-initialise `w` to 0, and the next inner loop iteration goes wrong.

From this analysis we can see that PPC is more powerful than the simple "execute each loop zero and one times" strategy in its ability to find bug.